

Machine Learning

Assignment 1: Clustering and Outlier Detection

Chia-Hsuan Hsu Student ID: 315511079

Fall 2026

Google Colab Links

- Part II: <https://colab.research.google.com/drive/1MhXa338fuY-S2YiQqLsj9U-7jck0fvEF?usp=sharing>
- Part III: https://colab.research.google.com/drive/1Nxm1CHly4_BwRQXENahAuM2pjmYhPBfH?usp=sharing

1 Part I: Clustering and PCA by Hand

1.1 Why Centre?

Consider the data matrix

$$M = \begin{bmatrix} 4 & 2 \\ 2 & 4 \\ 3 & 3 \\ 3 & 3 \end{bmatrix}.$$

1.1.1 Eigen-decomposition of $M^\top M$

First,

$$M^\top M = \begin{bmatrix} 4 & 2 & 3 & 3 \\ 2 & 4 & 3 & 3 \end{bmatrix} \begin{bmatrix} 4 & 2 \\ 2 & 4 \\ 3 & 3 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 38 & 34 \\ 34 & 38 \end{bmatrix}.$$

The characteristic equation is

$$\det \begin{bmatrix} 38 - \lambda & 34 \\ 34 & 38 - \lambda \end{bmatrix} = 0,$$

so

$$(38 - \lambda)^2 - 34^2 = 0.$$

Therefore,

$$\lambda_1 = 72, \quad \lambda_2 = 4.$$

For $\lambda_1 = 72$, an eigenvector is

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

so the corresponding unit eigenvector is

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

For $\lambda_2 = 4$, an eigenvector is

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

so the corresponding unit eigenvector is

$$v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Thus, without centring, the leading direction is

$$\boxed{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}}.$$

1.1.2 Centring the data

The mean of each column is

$$\mu_1 = \mu_2 = \frac{4 + 2 + 3 + 3}{4} = 3.$$

Therefore, the centred matrix is

$$\widetilde{M} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

We then obtain

$$\widetilde{M}^\top \widetilde{M} = \begin{bmatrix} 2 & -2 \\ -2 & 2 \end{bmatrix}.$$

Using the covariance matrix specified in the question,

$$C = \frac{1}{4} \widetilde{M}^\top \widetilde{M} = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} \end{bmatrix}.$$

The eigenvalues of C are

$$\lambda_1 = 1, \quad \lambda_2 = 0.$$

For $\lambda_1 = 1$, the unit eigenvector is

$$u_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

while for $\lambda_2 = 0$ the unit eigenvector is

$$u_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Hence, after centring, the first principal component is

$$\boxed{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}}.$$

The leading eigenvector changes because $M^\top M$ is computed relative to the origin. Since all observations have a large common mean in the $(1, 1)$ direction, the uncentred matrix treats this displacement from the origin as an important direction. PCA is intended to describe variation around the data mean rather than the location of the mean itself. After centring, the common offset disappears, and the only non-zero variation is in the $(1, -1)$ direction. Therefore, PCA should report $\frac{1}{\sqrt{2}}(1, -1)^\top$ as the first principal component.

1.2 k -means by Hand

Consider the seven stall positions

$$\{1, 2, 3, 8, 9, 10, 25\},$$

with $k = 2$ and squared Euclidean cost

$$\phi = \sum_x \min_{c \in C} (x - c)^2.$$

1.2.1 Initialisation A: centroids $\{1, 25\}$

At iteration 0, the centroids are

$$C^{(0)} = \{1, 25\}.$$

The first six points are closer to 1, while 25 is assigned to the second cluster. Therefore,

$$C_1 = \{1, 2, 3, 8, 9, 10\}, \quad C_2 = \{25\}.$$

The initial cost is

$$\begin{aligned} \phi^{(0)} &= (1 - 1)^2 + (2 - 1)^2 + (3 - 1)^2 + (8 - 1)^2 \\ &\quad + (9 - 1)^2 + (10 - 1)^2 + (25 - 25)^2 \\ &= 0 + 1 + 4 + 49 + 64 + 81 + 0 \\ &= 199. \end{aligned}$$

Updating the centroids gives

$$c_1 = \frac{1 + 2 + 3 + 8 + 9 + 10}{6} = 5.5, \quad c_2 = 25.$$

The assignments do not change. The new cost is

$$\begin{aligned} \phi^{(1)} &= (1 - 5.5)^2 + (2 - 5.5)^2 + (3 - 5.5)^2 \\ &\quad + (8 - 5.5)^2 + (9 - 5.5)^2 + (10 - 5.5)^2 \\ &= 77.5. \end{aligned}$$

Since another assignment and update step produces the same clusters and centroids, the algorithm has converged.

Table 1: k -means iterations for initialisation A.

Iteration	Centroids	Cluster 1	Cluster 2	ϕ
0	(1, 25)	$\{1, 2, 3, 8, 9, 10\}$	$\{25\}$	199
1	(5.5, 25)	$\{1, 2, 3, 8, 9, 10\}$	$\{25\}$	77.5

1.2.2 Initialisation B: centroids $\{1, 2\}$

Now let

$$C^{(0)} = \{1, 2\}.$$

Using the smaller centroid index to break any ties, the initial assignments are

$$C_1 = \{1\}, \quad C_2 = \{2, 3, 8, 9, 10, 25\}.$$

The initial cost is

$$\begin{aligned} \phi^{(0)} &= (1 - 1)^2 + (2 - 2)^2 + (3 - 2)^2 + (8 - 2)^2 \\ &\quad + (9 - 2)^2 + (10 - 2)^2 + (25 - 2)^2 \\ &= 679. \end{aligned}$$

Updating the centroids gives

$$c_1 = 1, \quad c_2 = \frac{2 + 3 + 8 + 9 + 10 + 25}{6} = 9.5.$$

With centroids $(1, 9.5)$, the new nearest-centroid assignments become

$$C_1 = \{1, 2, 3\}, \quad C_2 = \{8, 9, 10, 25\}.$$

The cost under these centroids is

$$\begin{aligned} \phi^{(1)} &= (1 - 1)^2 + (2 - 1)^2 + (3 - 1)^2 \\ &\quad + (8 - 9.5)^2 + (9 - 9.5)^2 + (10 - 9.5)^2 + (25 - 9.5)^2 \\ &= 248. \end{aligned}$$

Updating again gives

$$c_1 = \frac{1 + 2 + 3}{3} = 2, \quad c_2 = \frac{8 + 9 + 10 + 25}{4} = 13.$$

The assignments remain

$$C_1 = \{1, 2, 3\}, \quad C_2 = \{8, 9, 10, 25\},$$

and the cost becomes

$$\begin{aligned} \phi^{(2)} &= (1 - 2)^2 + (2 - 2)^2 + (3 - 2)^2 \\ &\quad + (8 - 13)^2 + (9 - 13)^2 + (10 - 13)^2 + (25 - 13)^2 \\ &= 196. \end{aligned}$$

The assignments and centroids no longer change, so the algorithm has converged.

Table 2: k -means iterations for initialisation B.

Iteration	Centroids	Cluster 1	Cluster 2	ϕ
0	(1, 2)	{1}	{2, 3, 8, 9, 10, 25}	679
1	(1, 9.5)	{1, 2, 3}	{8, 9, 10, 25}	248
2	(2, 13)	{1, 2, 3}	{8, 9, 10, 25}	196

Initialisation A reaches the lower final cost:

$$77.5 < 196.$$

In one dimension, an optimal k -means partition can be taken to consist of contiguous groups. Checking the possible split locations shows that the split

$$\{1, 2, 3, 8, 9, 10\} \quad \text{and} \quad \{25\}$$

has the smallest cost, 77.5. Therefore the solution obtained from initialisation B is not a global optimum.

The reason the two initialisations produce different solutions is that the k -means objective is non-convex. Lloyd's algorithm repeatedly improves the current assignments and centroids, but it is only guaranteed to converge to a local optimum, which can depend strongly on the initial centroids.

1.2.3 Farthest-first initialisation

Starting with position 1, the point farthest from the first centroid is 25. Therefore, for $k = 2$, farthest-first chooses

$$\boxed{\{1, 25\}}.$$

For $k = 3$, after selecting 1 and 25, the distance of each remaining point to its nearest selected centroid is

x	2	3	8	9	10
$\min_{c \in \{1, 25\}} x - c $	1	2	7	8	9

so the third centroid is 10. Thus,

$$\boxed{\{1, 25, 10\}}$$

is selected for $k = 3$.

The isolated point at 25 is immediately selected because it is much farther from the other observations than any ordinary point. This illustrates that farthest-first initialization is highly sensitive to outliers: an extreme observation can consume a centroid even when it does not represent a dense or useful cluster.

1.3 Hierarchical Clustering by Hand

Consider the five cart positions

$$\{0, 2, 3, 7, 12\},$$

with distance

$$d(x, y) = |x - y|.$$

1.3.1 Single-link clustering

Single link defines the distance between two clusters as the smallest distance between any pair of their members.

Initially, the nearest pair is 2 and 3:

$$\{2\}, \{3\} \longrightarrow \{2, 3\}, \quad h = 1.$$

The distance from $\{0\}$ to $\{2, 3\}$ is

$$\min\{|0 - 2|, |0 - 3|\} = 2,$$

so the next merge is

$$\{0\}, \{2, 3\} \longrightarrow \{0, 2, 3\}, \quad h = 2.$$

The distance from $\{0, 2, 3\}$ to $\{7\}$ is

$$\min\{7, 5, 4\} = 4,$$

so

$$\{0, 2, 3\}, \{7\} \longrightarrow \{0, 2, 3, 7\}, \quad h = 4.$$

Finally,

$$d(\{0, 2, 3, 7\}, \{12\}) = \min\{12, 10, 9, 5\} = 5,$$

so the last merge occurs at height 5.

Table 3: Single-link agglomerative clustering.

Step	Merge	Height
1	$\{2\}$ with $\{3\}$	1
2	$\{0\}$ with $\{2, 3\}$	2
3	$\{0, 2, 3\}$ with $\{7\}$	4
4	$\{0, 2, 3, 7\}$ with $\{12\}$	5

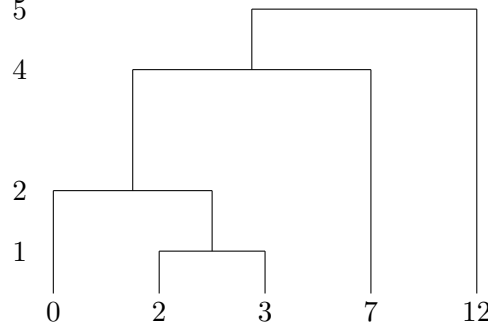


Figure 1: Single-link dendrogram. Merge heights are shown on the vertical axis.

Cutting the dendrogram into two clusters gives

$$\boxed{\{0, 2, 3, 7\}} \quad \text{and} \quad \boxed{\{12\}}.$$

Thus, the cart at 7 belongs to the cluster containing 0, 2, and 3.

1.3.2 Complete-link clustering

Complete link defines the distance between two clusters as the largest distance between any pair of their members.

As before, the first merge is

$$\{2\}, \{3\} \longrightarrow \{2, 3\}, \quad h = 1.$$

The complete-link distance from $\{0\}$ to $\{2, 3\}$ is

$$\max\{|0 - 2|, |0 - 3|\} = 3.$$

Therefore,

$$\{0\}, \{2, 3\} \longrightarrow \{0, 2, 3\}, \quad h = 3.$$

The distance between 7 and 12 is

$$|7 - 12| = 5,$$

which is smaller than the complete-link distance between $\{0, 2, 3\}$ and $\{7\}$:

$$\max\{7, 5, 4\} = 7.$$

Hence,

$$\{7\}, \{12\} \longrightarrow \{7, 12\}, \quad h = 5.$$

Finally, the complete-link distance between $\{0, 2, 3\}$ and $\{7, 12\}$ is

$$\max\{7, 12, 5, 10, 4, 9\} = 12.$$

Thus the final merge occurs at height 12.

Table 4: Complete-link agglomerative clustering.

Step	Merge	Height
1	$\{2\}$ with $\{3\}$	1
2	$\{0\}$ with $\{2, 3\}$	3
3	$\{7\}$ with $\{12\}$	5
4	$\{0, 2, 3\}$ with $\{7, 12\}$	12

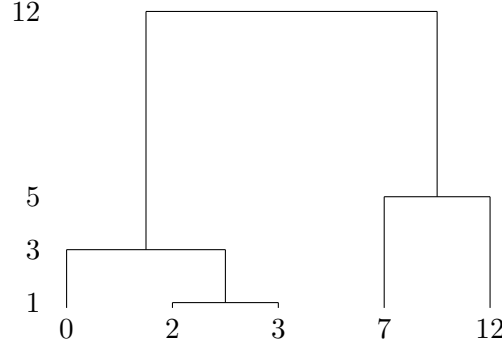


Figure 2: Complete-link dendrogram. Merge heights are shown on the vertical axis.

Cutting the complete-link dendrogram into two clusters gives

$$\boxed{\{0, 2, 3\}} \quad \text{and} \quad \boxed{\{7, 12\}}.$$

Therefore, the cart at 7 is grouped with 12 under complete linkage.

This example illustrates the *chaining effect* of single linkage. Single linkage only requires one close pair of points to connect two clusters, so a sequence of moderately close observations can progressively join distant regions into one long cluster. Here, the point at 7 connects to $\{0, 2, 3\}$ through the relatively short distance from 3 to 7, and 12 then connects through the distance from 7 to 12. Complete linkage instead considers the farthest pair of points and therefore tends to form more compact clusters.

1.4 DBSCAN by Hand

Return to the seven stall positions

$$\{1, 2, 3, 8, 9, 10, 25\},$$

with

$$\varepsilon = 1, \quad \text{MinPts} = 3.$$

The ε -neighbourhood includes the point itself.

The neighbourhoods are

$$\begin{aligned} N_\varepsilon(1) &= \{1, 2\}, \\ N_\varepsilon(2) &= \{1, 2, 3\}, \\ N_\varepsilon(3) &= \{2, 3\}, \\ N_\varepsilon(8) &= \{8, 9\}, \\ N_\varepsilon(9) &= \{8, 9, 10\}, \\ N_\varepsilon(10) &= \{9, 10\}, \\ N_\varepsilon(25) &= \{25\}. \end{aligned}$$

A point is a core point when its neighbourhood contains at least three points. Therefore, 2 and 9 are core points.

Points 1 and 3 are within distance ε of the core point 2, so they are border points. Similarly, 8 and 10 are border points because they are within distance ε of the core point 9.

The point 25 is neither a core point nor within the neighbourhood of a core point, so it is noise.

Table 5: DBSCAN classification with $\varepsilon = 1$ and MinPts = 3.

Stall	Type	ε -neighbourhood
1	Border	$\{1, 2\}$
2	Core	$\{1, 2, 3\}$
3	Border	$\{2, 3\}$
8	Border	$\{8, 9\}$
9	Core	$\{8, 9, 10\}$
10	Border	$\{9, 10\}$
25	Noise	$\{25\}$

The resulting DBSCAN clusters are

$$\boxed{\{1, 2, 3\}}, \quad \boxed{\{8, 9, 10\}},$$

with

$$\boxed{\{25\}}$$

classified as noise.

Compared with k -means, DBSCAN handles the isolated stall at 25 more naturally. k -means must assign every observation to a cluster, so the outlying point either becomes its own cluster, as in initialisation A, or substantially distorts the centroid of a larger cluster, as in initialisation B. DBSCAN can explicitly label 25 as noise while retaining the two dense groups $\{1, 2, 3\}$ and $\{8, 9, 10\}$ as clusters.

2 Part II: k -means from Scratch

2.1 Data Preparation

The Spotify Tracks Dataset was filtered to retain only the four genres **classical**, **hip-hop**, **mandopop**, and **metal**. The original subset contained 4,000 tracks. Rows with duplicated track IDs were removed first, followed by rows whose nine audio features exactly duplicated an earlier observation. After both duplicate-removal steps, 3,408 tracks remained, matching the expected sanity check.

The nine audio features used were danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo. Each feature was standardized as

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

where σ_j is the population standard deviation (`ddof=0`). After standardization, every feature had mean approximately zero and standard deviation approximately one.

2.1.1 Dataset statistics

Table 6: Number of retained tracks by genre after duplicate removal.

Genre	Number of tracks
Classical	814
Hip-hop	747
Mandopop	985
Metal	862
Total	3408

Table 7: Skewness of the nine audio features.

Feature	Skewness
Danceability	-0.0039
Energy	-0.3102
Loudness	-1.6515
Speechiness	3.0426
Acousticness	0.3377
Instrumentalness	1.7102
Liveness	2.3003
Valence	0.3892
Tempo	0.2730

Speechiness and liveness are strongly right-skewed, while loudness is strongly left-skewed and instrumentalness is also substantially right-skewed. Strong skewness can create observations far from the centre of the distribution, and because distance-based methods depend on Euclidean distances, these extreme observations can have a large influence on centroid locations and nearest-neighbour relationships.

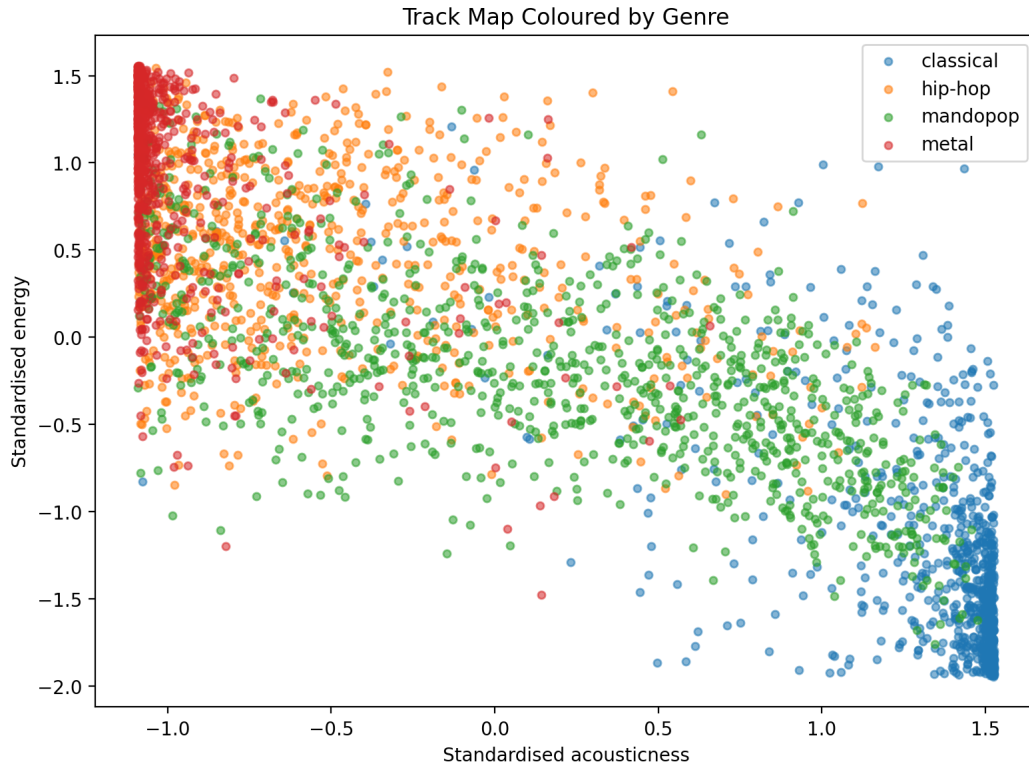


Figure 3: Track map showing standardized acousticness against standardized energy, coloured by genre.

Figure 3 shows a clear broad separation between classical and the higher-energy genres. Classical tracks are concentrated toward high acousticness and low energy, whereas metal is concentrated toward low acousticness and high energy. Hip-hop and mandopop occupy a wider middle region and overlap substantially with each other. Metal also overlaps with hip-hop and mandopop near the low-acousticness region, so the genre structure is not perfectly separated in this two-dimensional projection.

2.2 *k*-means Implementation

The function `kmeans(X, C0, max_iter=20)` was implemented from scratch using NumPy. For a set of centroids C , the objective is

$$\phi = \sum_x \min_{c \in C} \|x - c\|_2^2.$$

The implementation assigns every observation to its nearest centroid, with ties resolved by the smallest centroid index. Each non-empty centroid is then updated to the mean of its assigned observations. If a cluster becomes empty, its previous centroid is retained. Pairwise point-to-centroid distances are computed using vectorized NumPy operations, with no explicit Python loop over data points.

```
def squared_distances(X, C):
    X2 = np.sum(X * X, axis=1, keepdims=True)
    C2 = np.sum(C * C, axis=1)[None, :]
    return np.maximum(X2 + C2 - 2.0 * X @ C.T, 0.0)

def kmeans(X, C0, max_iter=20):
    C = np.asarray(C0, dtype=float).copy()
    n, k = X.shape[0], C.shape[0]
    D2 = squared_distances(X, C)
    assign = np.argmin(D2, axis=1) # ties -> smallest index
    history = [D2[np.arange(n), assign].sum()] # phi(0)
    for _ in range(max_iter):
        new_C = C.copy()
        for j in range(k): # loop over clusters, not points
            mask = assign == j
            if np.any(mask):
                new_C[j] = X[mask].mean(axis=0)
            # empty cluster: keep previous centroid
        C = new_C
        D2 = squared_distances(X, C)
        assign = np.argmin(D2, axis=1)
        history.append(D2[np.arange(n), assign].sum())
    return assign, C, np.asarray(history)
```

2.2.1 Verification against scikit-learn

To verify the implementation, the custom method and `sklearn.cluster.KMeans` were initialized with exactly the same four centroids and run for at most 20 iterations. The two final SSE values agree to numerical precision.

Table 8: Verification of the custom *k*-means implementation.

Method	Final SSE
Custom NumPy implementation	15821.134129
scikit-learn verification	15821.134129

The absolute difference between the two results was

$$3.64 \times 10^{-12},$$

confirming that the custom implementation reproduces the reference implementation to floating-point precision.

2.3 Initialisation Strategies

For $k = 4$, three initialization strategies were evaluated over seeds $0, \dots, 19$:

1. random initialization using four distinct observations,
2. farthest-first initialization,
3. k -means++ initialization.

For each seeded method, the first point was selected using `numpy.random.default_rng(seed).integers(n)` as specified in the assignment.

```
def init_farthest_first(X, k, seed):
    rng = np.random.default_rng(seed)
    chosen = [int(rng.integers(X.shape[0]))]
    min_d2 = squared_distances(X, X[[chosen[0]]][:, 0])
    while len(chosen) < k:
        nxt = int(np.argmax(min_d2)) # farthest point
        chosen.append(nxt)
        min_d2 = np.minimum(min_d2, squared_distances(X, X[[nxt]]][:, 0])
    return X[chosen].copy(), np.array(chosen)

def init_kmeans_pp(X, k, seed):
    rng = np.random.default_rng(seed)
    n = X.shape[0]
    chosen = [int(rng.integers(n))]
    min_d2 = squared_distances(X, X[[chosen[0]]][:, 0])
    while len(chosen) < k:
        p = min_d2 / min_d2.sum() # prob. proportional to D(x)^2
        nxt = int(rng.choice(n, p=p))
        chosen.append(nxt)
        min_d2 = np.minimum(min_d2, squared_distances(X, X[[nxt]]][:, 0])
    return X[chosen].copy(), np.array(chosen)
```

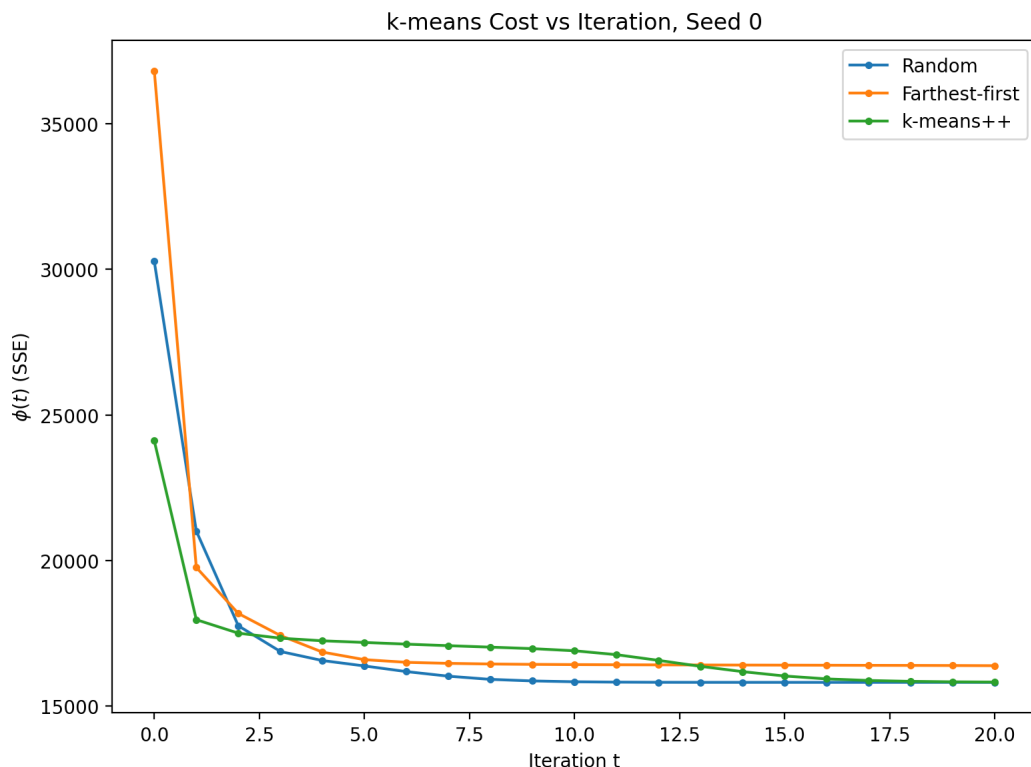


Figure 4: k -means cost $\phi(t)$ over iterations for seed 0 under the three initialization strategies.

All three methods reduce the objective rapidly during the first few iterations. However, their starting costs differ substantially, showing that initialization can strongly affect the starting configuration and the local optimum reached by Lloyd’s algorithm.

Table 9: Initial and final SSE across 20 seeds.

Strategy	Median $\phi(0)$	Min $\phi(20)$	Median $\phi(20)$	Max $\phi(20)$
Random	29350.87	15811.07	15818.53	17186.42
Farthest-first	44622.97	15824.77	16373.39	16458.67
k -means++	28192.53	15811.39	16157.34	17143.26

Among the three strategies, k -means++ had the lowest median initial cost, with median $\phi(0) = 28192.53$, so it provided the best starting configurations according to this criterion. Random initialization had the lowest median final cost, with median $\phi(20) = 15818.53$, and also obtained the lowest single final cost, 15811.07. In this experiment, therefore, the strategy that starts best is not necessarily the strategy with the best median final result. Farthest-first had by far the largest median initial cost and a higher median final cost than the other two methods.

2.3.1 Farthest-first selections for seed 0

The four observations selected by farthest-first initialization with seed 0 are shown in Table 10. For each track, the most extreme standardized feature is reported.

Table 10: Tracks selected by farthest-first initialization with seed 0.

Track	Artist	Genre	Extreme feature	z value
Wild World – 2009 Remastered Version	Mr. Big	metal	tempo	0.788
Sri Venkatesa Suprabhatam	M. S. Subbulakshmi; T. K. Murthy; V.V. Subramaniam	classical	speechiness	9.662
Glassworks: Opening	Philip Glass; Víkingur Ólafsson	classical	loudness	-4.164
What a Karavaad	Anirudh Ravichander; Dhanush	hip-hop	speechiness	4.959

After the first randomly chosen track, farthest-first repeatedly chooses the observation with the greatest distance from all previously selected centroids. Consequently, extremely unusual observations can be selected very early. For example, *Sri Venkatesa Suprabhatam* has a speechiness value about 9.66 standard deviations from the mean, while *Glassworks: Opening* has an unusually low loudness value of about -4.16 standard deviations. These selections demonstrate the sensitivity of farthest-first initialization to outliers: an isolated point may receive a centroid even if it does not represent a large, dense group of tracks.

2.4 Choosing k and Cluster Validity

For each

$$k = 2, \dots, 8,$$

k -means++ initialization was run using seeds $0, \dots, 9$, and the run with the lowest final SSE was retained.

Table 11: Best k -means result and validity measures for each k .

k	Best SSE	Silhouette	ARI
2	21055.40	0.3648	0.1981
3	17911.71	0.2020	0.3507
4	15811.39	0.2087	0.4699
5	14417.40	0.2103	0.3324
6	13033.22	0.2142	0.3472
7	12280.04	0.1988	0.2968
8	11556.83	0.1858	0.2454

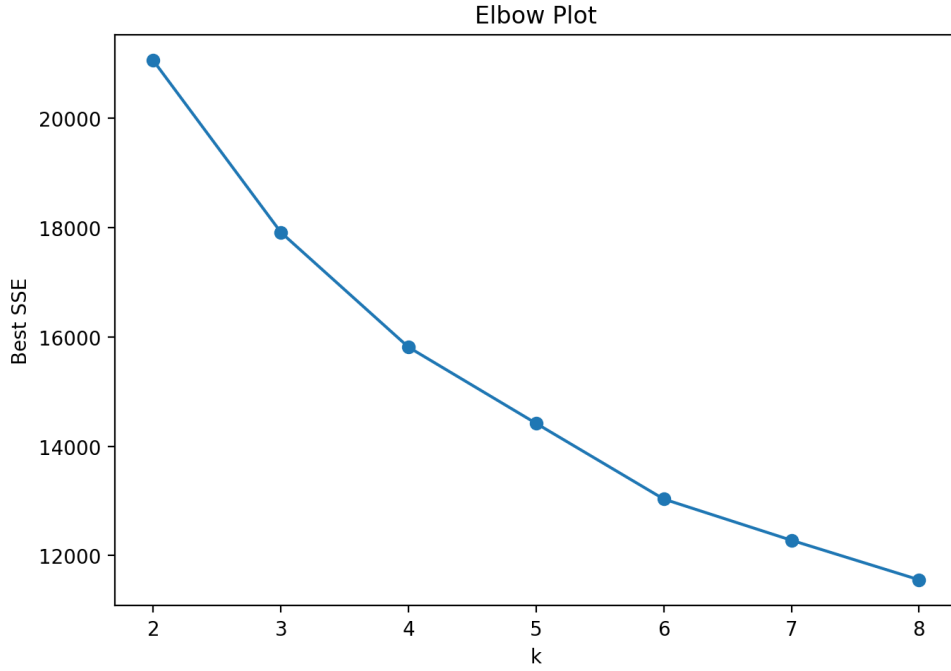


Figure 5: Best SSE as a function of k .

The SSE decreases monotonically as k increases, as expected because additional centroids give the model more flexibility. The largest reductions occur at the smaller values of k , and the curve begins to flatten around $k = 4$ – 6 . Thus, the elbow criterion does not give a perfectly sharp choice, but $k = 4$ is a reasonable parsimonious point after the steep initial decrease.

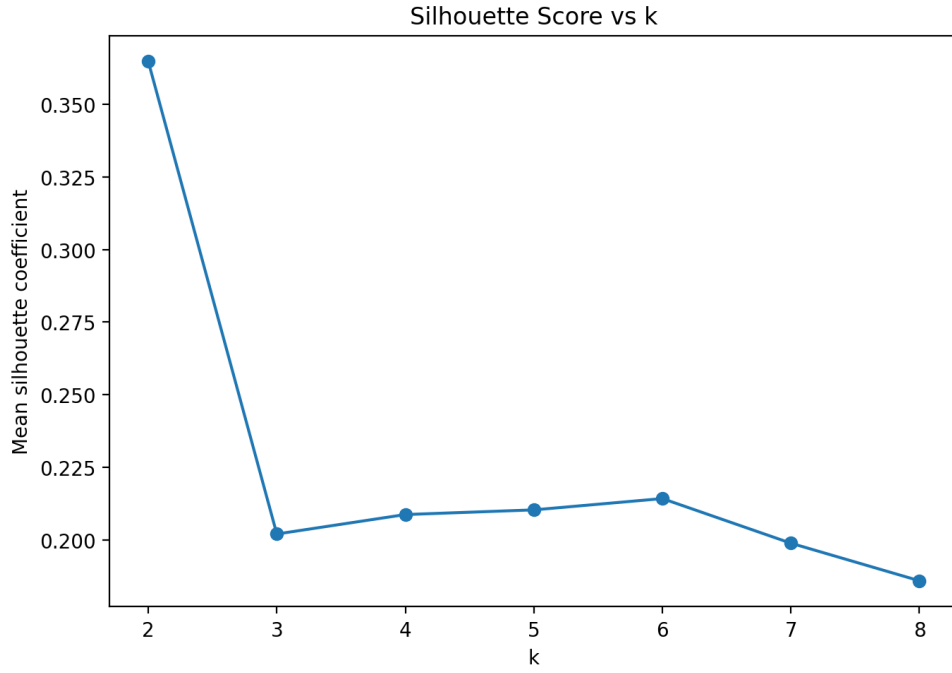


Figure 6: Mean silhouette coefficient as a function of k .

The silhouette coefficient is largest at

$$k = 2,$$

with a value of approximately 0.3648. Thus, according to the internal geometry of the feature space, a two-cluster partition gives the strongest separation between compact groups.

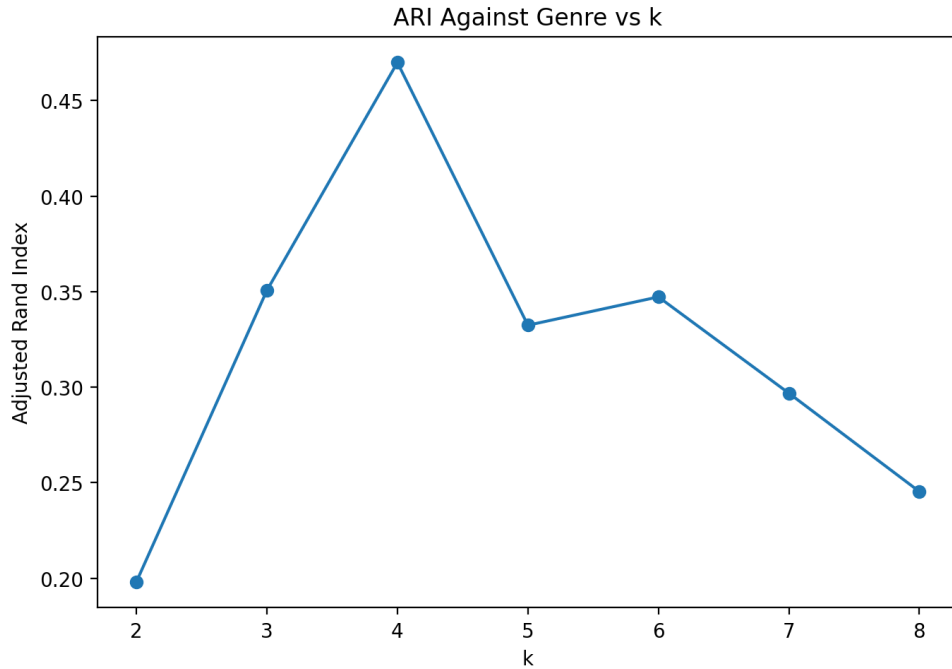


Figure 7: Adjusted Rand index against genre as a function of k .

In contrast, the adjusted Rand index is largest at

$$k = 4,$$

with

$$\text{ARI} \approx 0.4699.$$

The silhouette coefficient and ARI favour different values of k because they measure different properties. Silhouette is an internal criterion: it uses only distances between observations and evaluates within-cluster compactness and between-cluster separation. ARI is an external criterion: it compares the cluster assignments with the known genre labels. Therefore, a clustering can form geometrically distinct groups without reproducing the four genre categories. Here, the strong silhouette result at $k = 2$ suggests that the most prominent geometric division in the audio-feature space is coarser than the four genre labels, whereas the maximum ARI at $k = 4$ indicates that four clusters agree best with the genre annotation.

For the night-market application, I would use four playlists. Although the silhouette score prefers two broad groups, $k = 4$ gives the highest agreement with the four known genres and lies near the early flattening region of the SSE curve. Four playlists therefore retain more musical variation than the two-cluster solution while avoiding unnecessary fragmentation into many small groups.

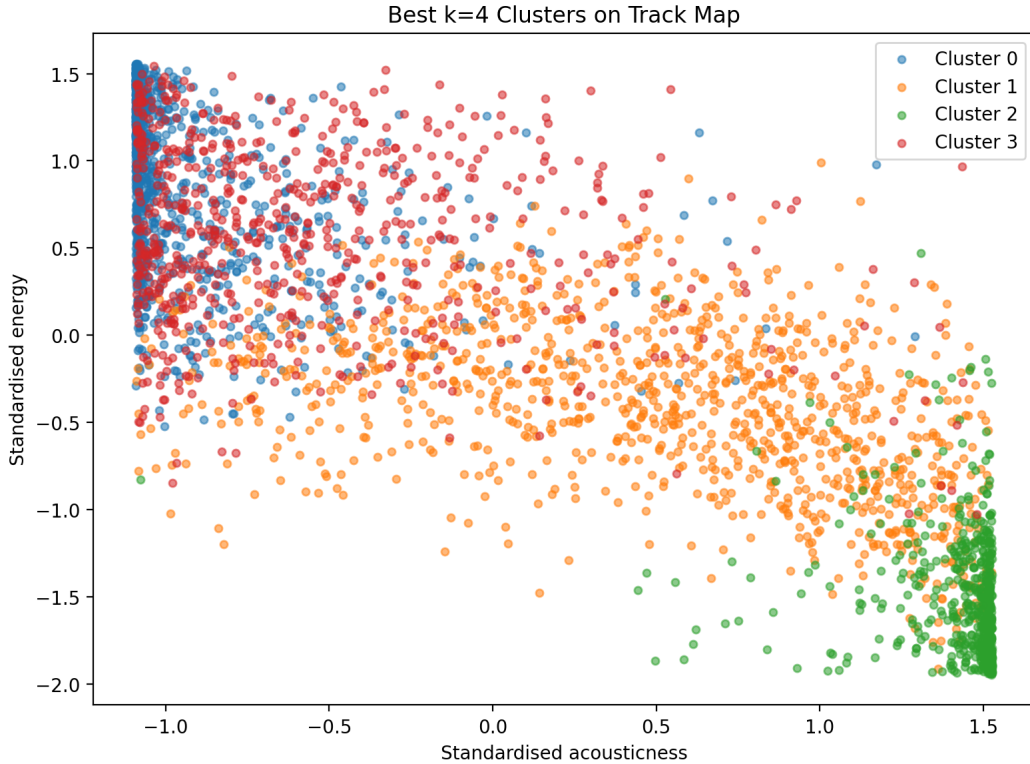


Figure 8: Best $k = 4$ clustering shown on the track map. The best run used seed 3 and achieved an SSE of 15811.39.

The $k = 4$ track map shows that the clusters correspond to broad regions of the acousticness–energy plane rather than reproducing genre labels exactly. A high-acousticness, low-energy region and a low-acousticness, high-energy region are especially visible, while many observations in the intermediate region are divided using information from the full nine-dimensional feature space.

2.4.1 Null-data experiment

To test whether the observed clustering structure depends on relationships among features, 10 null datasets were created by independently permuting each column of Z . This preserves the marginal distribution of each feature but destroys dependence among features. For each null dataset and every $k = 2, \dots, 8$, the same k -means++ procedure was repeated.

Table 12: Best SSE on the real data and mean best SSE across the 10 null datasets.

k	Real SSE	Mean null SSE	Null SD
2	21055.40	27558.90	213.53
3	17911.71	25118.79	165.67
4	15811.39	23021.08	84.05
5	14417.40	21395.39	172.16
6	13033.22	19825.27	132.84
7	12280.04	18821.67	212.81
8	11556.83	17982.14	52.27

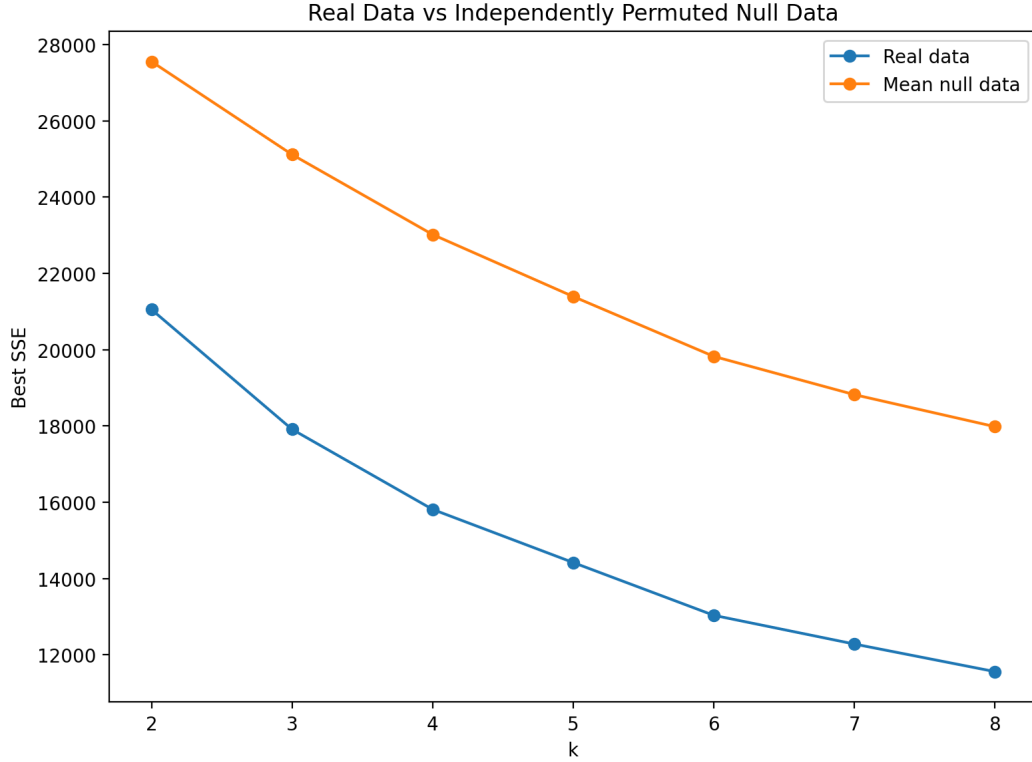


Figure 9: Best SSE on the real data compared with mean SSE from independently permuted null datasets.

For every value of k , the real data achieve a substantially lower SSE than the independently permuted null datasets. For example, at $k = 4$, the real-data SSE is approximately 15811, whereas the mean null SSE is approximately 23021. Because the null datasets preserve each feature’s individual distribution but remove their cross-feature relationships, this gap indicates that the real Spotify data contain meaningful multivariate structure. The clustering is therefore not explained merely by the marginal distributions of individual audio features.

3 Part III: Outlier Detection

3.1 Warm-up by Hand

3.1.1 z -score and masking effect

Consider again the seven stall positions

$$\{1, 2, 3, 8, 9, 10, 25\}.$$

Using the maximum-likelihood estimates, the mean is

$$\mu = \frac{1 + 2 + 3 + 8 + 9 + 10 + 25}{7} = \frac{58}{7} \approx 8.2857.$$

The maximum-likelihood standard deviation is

$$\sigma = \sqrt{\frac{1}{7} \sum_{i=1}^7 (x_i - \mu)^2} \approx 7.5916.$$

Therefore, the z -score of the stall at 25 is

$$z = \frac{25 - \mu}{\sigma} = \frac{25 - 8.2857}{7.5916} \approx 2.2017.$$

Thus,

$$|z| \approx 2.20 < 3,$$

so the stall at 25 is *not* flagged by the rule $|z| > 3$.

This illustrates the masking effect. The extreme observation at 25 itself increases both the estimated mean and the estimated standard deviation. As a result, its standardized distance from the mean becomes smaller than it would have been if the distributional parameters had been estimated from only the regular observations. An outlier can therefore partially hide its own extremeness.

3.1.2 Local Outlier Factor with $k = 2$

For each point o , the two nearest neighbours $N_2(o)$ and the distance to the second nearest neighbour $\text{dist}_2(o)$ are first determined. The local reachability density and LOF are then computed from

$$\begin{aligned} \text{reach}_2(o \leftarrow o') &= \max\{\text{dist}_2(o'), d(o, o')\}, \\ \text{lrd}_2(o) &= \left(\frac{1}{2} \sum_{o' \in N_2(o)} \text{reach}_2(o \leftarrow o') \right)^{-1}, \end{aligned}$$

and

$$\text{LOF}_2(o) = \frac{1}{2} \sum_{o' \in N_2(o)} \frac{\text{lrd}_2(o')}{\text{lrd}_2(o)}.$$

The results are shown in Table 13.

Table 13: LOF calculations for the seven stalls with $k = 2$.

Stall	dist_2	$N_2(o)$	lrd_2	LOF_2
1	2	$\{2, 3\}$	0.6667	0.8750
2	1	$\{1, 3\}$	0.5000	1.3333
3	2	$\{1, 2\}$	0.6667	0.8750
8	2	$\{9, 10\}$	0.6667	0.8750
9	1	$\{8, 10\}$	0.5000	1.3333
10	2	$\{8, 9\}$	0.6667	0.8750
25	16	$\{9, 10\}$	0.0645	9.0417

The stall at 25 has by far the largest LOF and is therefore the strongest outlier. The points 2 and 9 also have LOF values greater than one even though they lie inside the two dense groups. This happens because LOF is a local density ratio rather than a binary outlier rule: small differences in local neighbourhood density can produce values above one. Therefore, a threshold should not be chosen simply as $\text{LOF} > 1$; a larger threshold such as 1.5 is commonly more useful for identifying clearly unusual observations.

3.2 Proximity-Based Approaches

The k NN-distance and Local Outlier Factor detectors were implemented from scratch using NumPy. To avoid constructing an $n \times n \times 9$ broadcast array, pairwise Euclidean distances were computed using

$$\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b.$$

For each observation, the k NN score is the distance to its k th nearest neighbour. LOF was implemented directly from the reachability distance, local reachability density, and LOF definitions.

```
def pairwise_distances_numpy(X):
    sq = np.sum(X * X, axis=1)
    D2 = sq[:, None] + sq[None, :] - 2.0 * (X @ X.T)
    return np.sqrt(np.maximum(D2, 0.0))

def knn_score(X, k):
    D = pairwise_distances_numpy(X)
    np.fill_diagonal(D, np.inf) # exclude self
    return np.partition(D, k - 1, axis=1)[:, k - 1]

def lof(X, k):
    n = X.shape[0]
    D = pairwise_distances_numpy(X)
    Dns = D.copy()
    np.fill_diagonal(Dns, np.inf) # exclude self
    nbr = np.argpartition(Dns, k - 1, axis=1)[:, :k] # N_k(o)
    kdist = np.partition(Dns, k - 1, axis=1)[:, k - 1] # dist_k(o)
    d_on = D[np.arange(n)[:, None], nbr] # d(o, o')
    reach = np.maximum(kdist[nbr], d_on) # reach_k(o <- o')
    lrd = 1.0 / reach.mean(axis=1)
    return lrd[nbr].mean(axis=1) / lrd
```

3.2.1 LOF verification

The custom LOF implementation was checked against `sklearn.neighbors.LocalOutlierFactor` with $k = 20$. Exact duplicate feature vectors had already been removed during Part II. This is important because exact duplicates can give zero neighbour and reachability distances, producing infinite or undefined local reachability densities and making LOF numerically unstable.

The maximum absolute difference between the custom LOF scores and scikit-learn was

$$1.09 \times 10^{-10},$$

which is below the required tolerance of 10^{-8} .

3.2.2 LOF results

A total of 38 tracks had $\text{LOF}_{20} > 1.5$. The largest LOF_{20} score was approximately

$$2.5693,$$

which is consistent with the expected sanity check.

Table 14: Five tracks with the largest LOF_{20} scores.

Rank	Track	Artist	Genre	LOF_{20}
1	12 Variations on 'La belle Françoise' in E flat, K.353: 8. Variation VII	Wolfgang Amadeus Mozart; Danielle Laval	classical	2.5693
2	Sri Venkatesa Suprabhatam	M. S. Subbulakshmi; T. K. Murthy; V.V. Subramaniam	classical	2.1400
3	3 Gymnopédies: No. 1 Lent et douloureux	Erik Satie; Alice Sara Ott	classical	1.9916
4	Lakshmi Ashtothram	M. S. Subbulakshmi	classical	1.9499
5	2 Canons in F for 2 voices in 1, K508a: Nos. 1-2	Wolfgang Amadeus Mozart; Karl-Heinz Steffens; Werner Mittelbach; Joachim Olszewski	classical	1.9268

The five highest-scoring tracks are all from the classical subset. This does not mean that classical music in general is anomalous. Rather, these particular tracks lie in locally sparse regions of the nine-dimensional standardized feature space relative to their nearest neighbours.

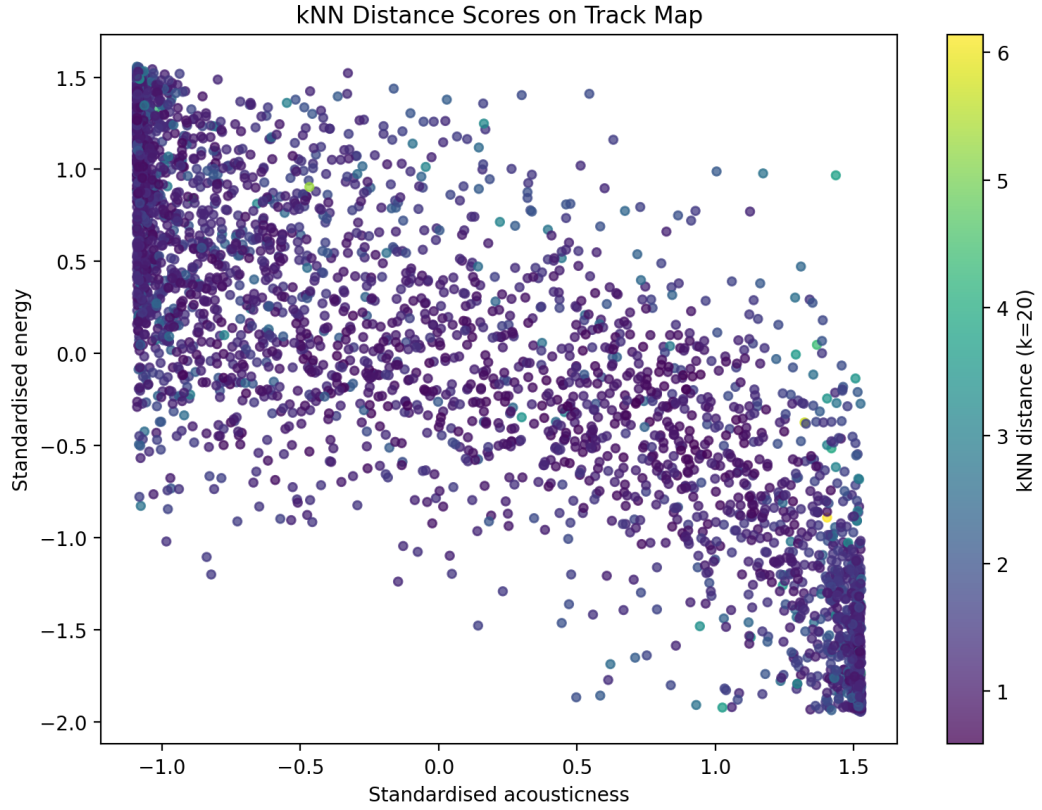


Figure 10: k NN distance scores with $k = 20$ shown as colour intensity on the track map.

The k NN score becomes large for observations that are far from their 20th nearest neighbour. Consequently, points in sparse regions and isolated parts of the feature space receive larger scores.

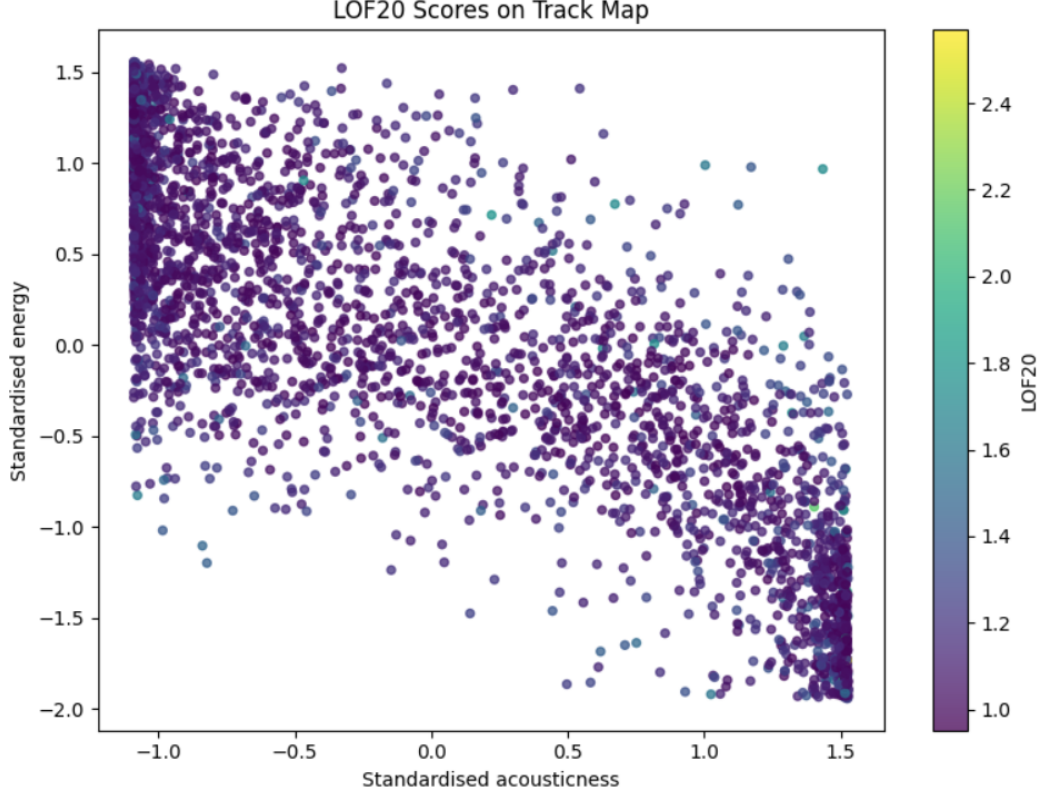


Figure 11: LOF₂₀ scores shown as colour intensity on the track map.

Unlike the kNN score, high LOF₂₀ values are not concentrated at the edges of the map. Points with large LOF appear scattered across the plane, including inside populated regions, because LOF measures density relative to each point’s own neighbours rather than absolute isolation. Points that look ordinary in this two-dimensional projection can still be locally sparse in the full nine-dimensional feature space.

3.3 Clustering-Based and Isolation Approaches

Using the best $k = 4$ solution from Part II, each observation was assigned the clustering-based outlier score

$$s(o) = \frac{d(o, c_o)}{\bar{d}(c_o)},$$

where c_o is the centroid of the cluster containing o and $\bar{d}(c_o)$ is the mean distance of that cluster’s members to the centroid.

DBSCAN was run with

$$\varepsilon = 2.0, \quad \text{min_samples} = 18,$$

and produced one cluster together with

$$\boxed{55}$$

noise points, matching the assignment sanity check.

An Isolation Forest with 200 trees and `random_state=0` was also fitted, using the negative of `score_samples` as its outlier score.

To compare the scoring methods with DBSCAN, the top

$$m = 55$$

observations under each score were marked as outliers, so that every scoring detector flagged the same number of observations as DBSCAN.

Table 15: Overlap between the top 55 scores from each detector and the 55 DBSCAN noise points.

Detector	Overlap with DBSCAN noise
k NN distance	46 / 55
LOF ₂₀	24 / 55
Clustering-based	33 / 55
Isolation Forest	30 / 55

The k NN-distance detector agrees most strongly with DBSCAN, sharing 46 of the 55 flagged points. This is expected because both methods are closely related to local neighbourhood sparsity. DBSCAN labels an observation as noise when it cannot be connected to a sufficiently dense neighbourhood, while the k NN score directly measures how far a point must reach to find its 20th neighbour.

The clustering-based score has the second-largest overlap with DBSCAN at 33 points, followed by Isolation Forest with 30. LOF has the smallest overlap, at 24 points. This lower overlap does not necessarily imply poorer outlier detection: LOF compares the density of a point with the density of its own neighbours and can therefore identify local density anomalies that are not globally isolated enough to be labelled as DBSCAN noise.

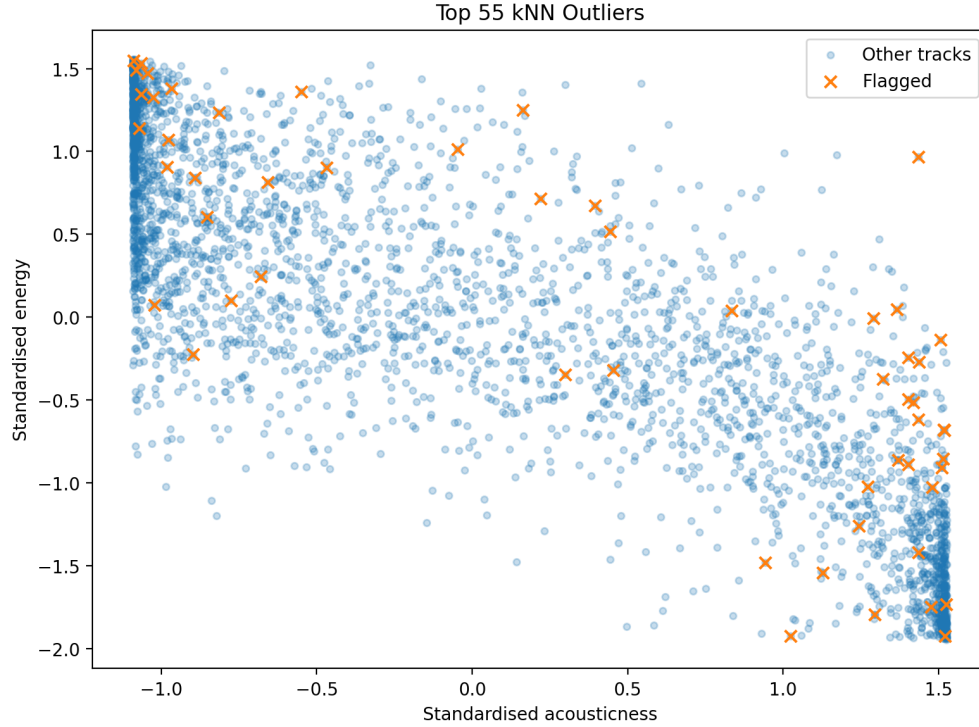


Figure 12: The 55 highest-scoring observations under the k NN distance detector.

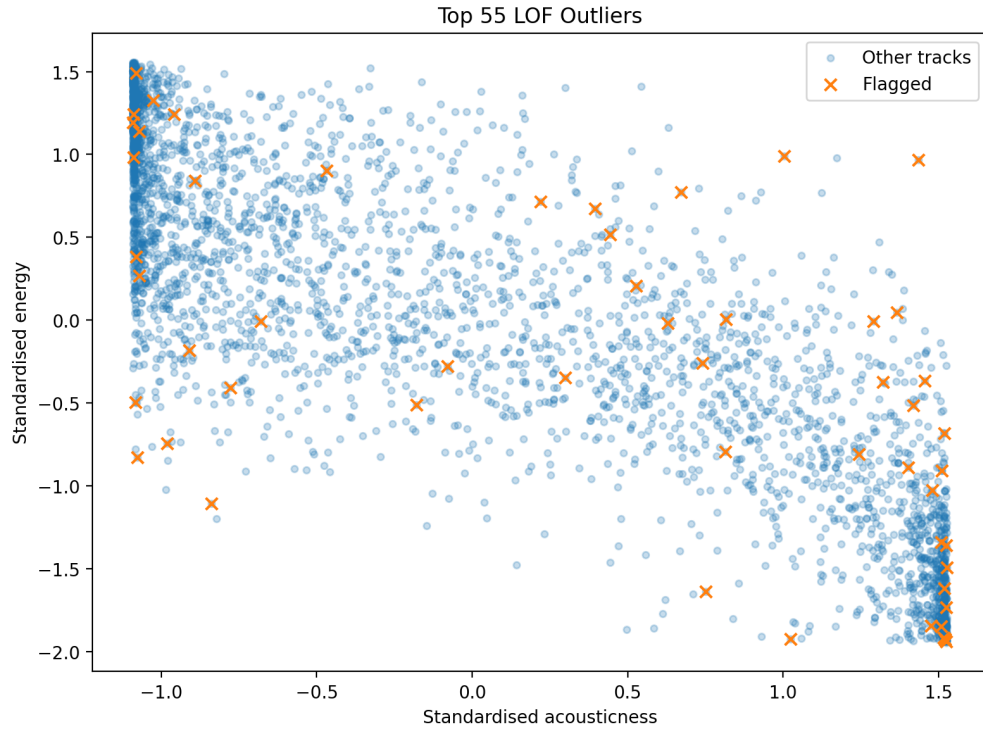


Figure 13: The 55 highest-scoring observations under LOF_{20} .

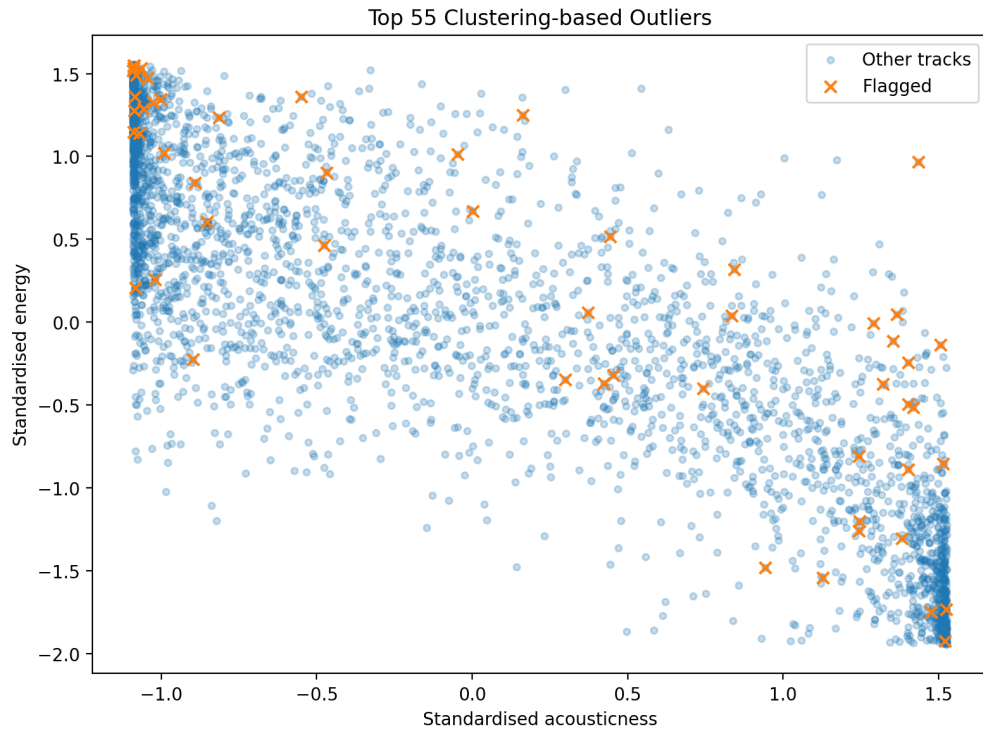


Figure 14: The 55 highest-scoring observations under the clustering-based detector.

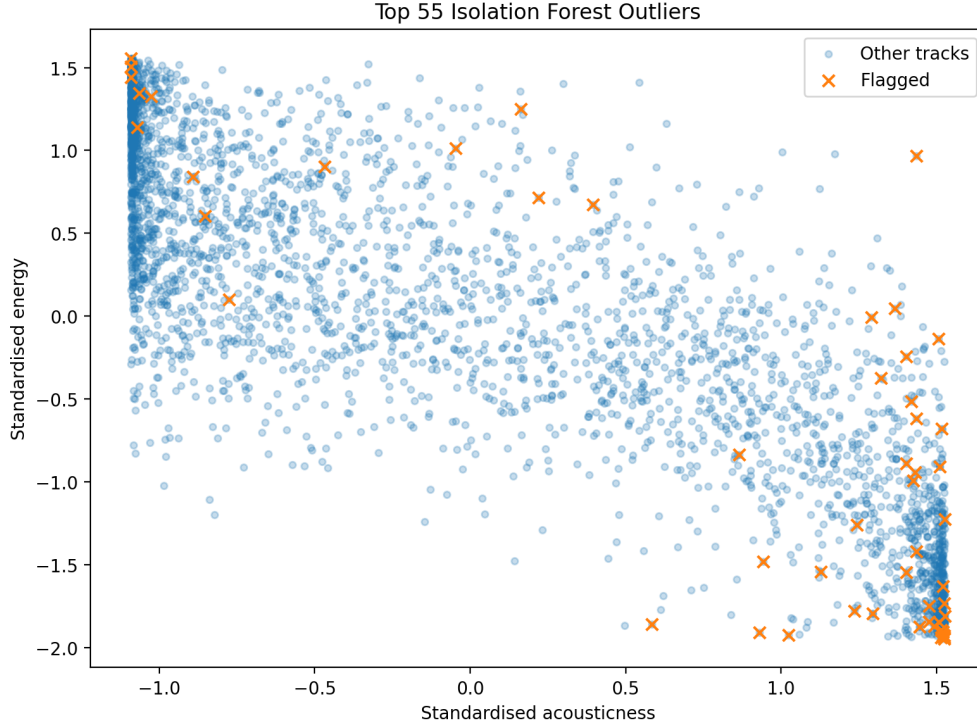


Figure 15: The 55 highest-scoring observations under Isolation Forest.

Several tracks among the DBSCAN noise points also receive very large LOF or k NN scores. For example, *12 Variations on ‘La belle Françoise’ in E flat, K.353: 8. Variation VII* has $\text{LOF}_{20} \approx 2.57$ and a k NN score of about 4.31. *Sri Venkatesa Suprabhatam* has $\text{LOF}_{20} \approx 2.14$ and an even larger k NN score of approximately 6.14. These observations are therefore unusual under several different notions of sparsity.

To judge whether flagged tracks are errors or genuinely unusual music, two of them were looked up.

- *Sri Venkatesa Suprabhatam* by M. S. Subbulakshmi is a well-known Sanskrit devotional hymn recited at dawn, chanted rather than sung as conventional melodic music. This explains its extreme speechiness ($z \approx 9.66$). It is *genuinely unusual music*, not a data error: it is labelled “classical” but is closer to spoken recitation than to typical classical recordings.
- *12 Variations on ‘La belle Françoise’, K.353: 8. Variation VII* is a short movement from Mozart’s solo piano variations. The recording itself is ordinary, but its combination of audio features has few close neighbours in this dataset, giving it the largest LOF_{20} (≈ 2.57). It is also *genuine music rather than an error*.

Other DBSCAN noise points, such as a live performance of Vivaldi’s *Concerto for 2 Cellos in G Minor, RV 531*, follow the same pattern: the flagged tracks are mostly legitimate recordings of unusual types (chanted, live, or very quiet music) rather than corrupted entries. For playlist screening, a flag should therefore prompt a manual check rather than automatic deletion.

3.4 Evaluation with Injected Anomalies

Two different sets of 20 synthetic anomalies were appended separately to the standardized Spotify data. In each experiment, the original tracks were labelled as normal and the injected observations were labelled as anomalies.

3.4.1 Global anomalies

The global anomalies were generated by drawing

$$U_i \sim N(0, I_9),$$

normalizing each row to unit length, and scaling each point by a radius drawn uniformly from 4 to 6. These observations are therefore placed far from the central region of the standardized feature space.

3.4.2 Correlation-breaking anomalies

The correlation-breaking anomalies were generated column by column. Each feature value was sampled from a value already observed in the corresponding column of Z . Therefore, every individual feature value is realistic, but the resulting combination of features may violate the dependencies present in real tracks.

For both augmented datasets, the k NN-distance, LOF₂₀, clustering-based, and Isolation Forest scoring detectors were re-fitted. The clustering-based detector also re-fitted k -means with $k = 4$. Performance was evaluated using ROC AUC and precision among the 20 highest-scoring observations.

Table 16: Performance of scoring detectors on injected anomalies.

Detector	Global anomalies		Correlation-breaking anomalies	
	ROC AUC	Precision@20	ROC AUC	Precision@20
k NN distance	0.9961	0.60	0.8117	0.15
LOF ₂₀	0.9993	0.85	0.8886	0.30
Clustering-based	0.9832	0.25	0.8340	0.05
Isolation Forest	0.9698	0.40	0.7189	0.05

All four scoring detectors perform very well on the global anomalies, with ROC AUC values above 0.95. LOF₂₀ gives the highest ROC AUC, 0.9993, and also the highest Precision@20, 0.85. The k NN-distance detector also performs strongly, with ROC AUC 0.9961.

The correlation-breaking anomalies are more difficult because each individual feature value is drawn from the real data and is therefore not extreme by itself. LOF₂₀ remains the strongest scoring method, with ROC AUC 0.8886 and Precision@20 0.30. The clustering-based method achieves ROC AUC 0.8340, followed by the k NN-distance method at 0.8117. Isolation Forest drops to ROC AUC 0.7189, indicating that it is less effective when anomalousness arises mainly from unusual feature combinations rather than globally extreme values.

Table 17: DBSCAN performance on injected anomalies.

Anomaly type	Precision	Recall	F1	Predicted noise	Injected detected
Global	0.2667	1.0000	0.4211	75	20 / 20
Correlation-breaking	0.0678	0.2000	0.1013	59	4 / 20

DBSCAN successfully detects all 20 global anomalies, giving recall 1.0. However, it also labels many original observations as noise, which reduces precision to 0.2667. For the correlation-breaking anomalies, DBSCAN detects only 4 of the 20 injected points, giving recall 0.20 and F1-score 0.1013. This occurs because correlation-breaking points may still lie inside regions that are sufficiently dense under DBSCAN’s fixed neighbourhood definition.

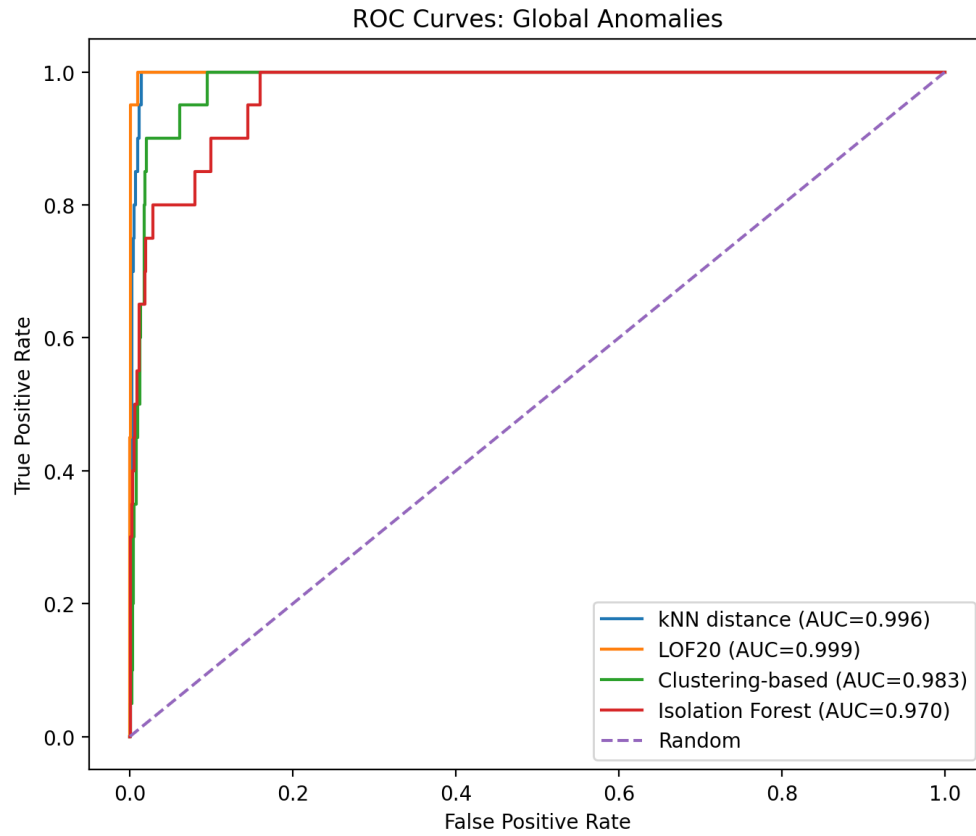


Figure 16: ROC curves for the global anomaly experiment.

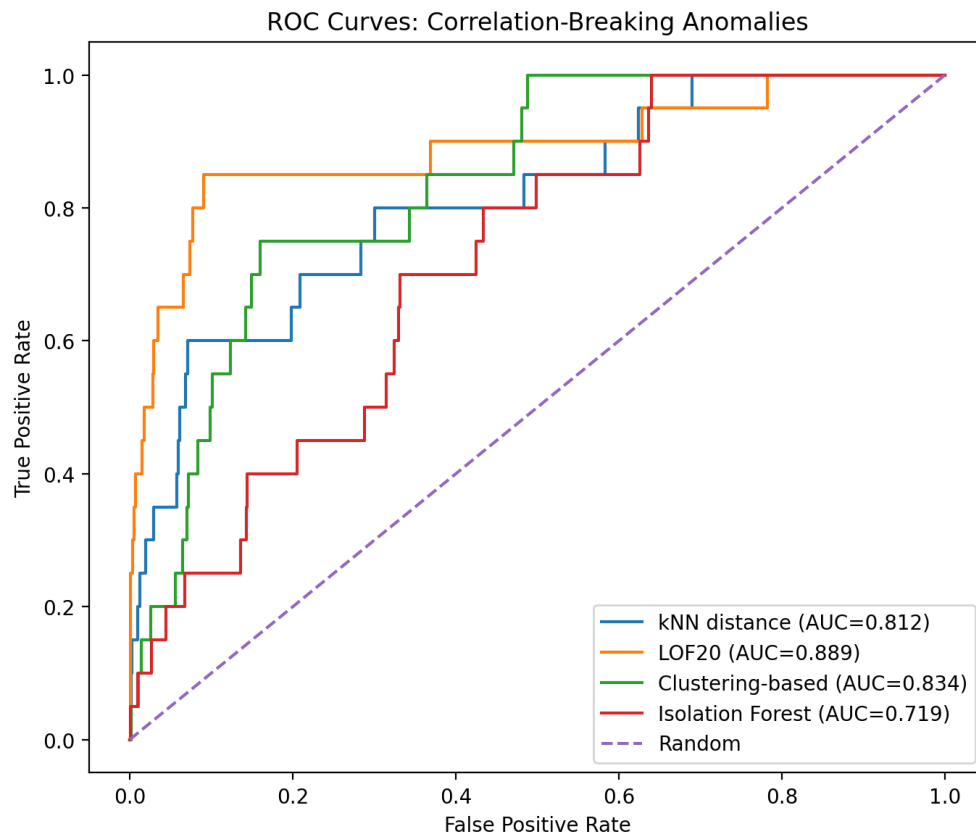


Figure 17: ROC curves for the correlation-breaking anomaly experiment.

3.4.3 Why accuracy is misleading

Accuracy is inappropriate for this experiment because the anomaly class is extremely rare. Each augmented dataset contains 3,408 normal tracks and only 20 planted anomalies. A classifier that predicts every observation as normal would therefore achieve

$$\frac{3408}{3428} \approx 0.9942,$$

or about 99.42% accuracy, despite detecting no anomalies at all. Metrics such as ROC AUC, precision, recall, and F1-score are therefore much more informative.

3.4.4 Comparison of detector assumptions

Global anomalies are far from the main body of data, so all four scoring methods detect them successfully. Both k NN distance and the clustering-based score respond directly to large geometric distances, while Isolation Forest can isolate such extreme observations with few tree splits. LOF also performs very strongly because these points have much lower local density than their neighbours.

Correlation-breaking anomalies are more subtle. Their marginal feature values are individually realistic, so methods that mainly respond to global extremeness lose effectiveness. LOF remains relatively strong because it compares each observation's local density with that of its neighbours, making it sensitive to unusual local combinations of features. This explains why LOF retains ROC AUC close to 0.89, while Isolation Forest falls to about 0.72.

3.4.5 Recommended detector

LOF₂₀ is the preferred detector for screening new tracks before they enter a playlist. It achieved the highest ROC AUC on both the global and correlation-breaking anomaly experiments and also obtained the highest Precision@20 in both cases. Its local-density comparison is particularly useful for detecting tracks whose individual features are not extreme but whose combination is unusual. For this dataset, LOF therefore provides the most consistent performance across both simple global anomalies and more subtle local anomalies.

4 Conclusion

This assignment examined clustering and outlier detection from both analytical and empirical perspectives. The hand calculations in Part I demonstrated why PCA must be performed on centred data, how k -means can converge to different local optima depending on initialization, how single-link hierarchical clustering can exhibit the chaining effect, and how DBSCAN can separate dense groups from isolated noise.

In Part II, k -means was implemented from scratch using NumPy and verified against scikit-learn to numerical precision. Among the three initialization strategies, k -means++ produced the lowest median initial SSE, while random initialization achieved the lowest median final SSE over the 20 seeds. The clustering validity criteria favoured different values of k : the silhouette coefficient was highest at $k = 2$, whereas the adjusted Rand index against the genre labels was highest at $k = 4$. The null-data experiment also produced substantially larger SSE values than the real Spotify data, indicating that the observed clustering reflects meaningful multivariate relationships between audio features rather than only their individual marginal distributions.

In Part III, proximity-based, clustering-based, density-based, and isolation-based outlier detectors were compared. DBSCAN identified 55 noise points in the original dataset, and the k NN-distance detector showed the greatest overlap with these points. The injected-anomaly experiments further showed that all four scoring detectors performed well on global anomalies, with ROC AUC values above 0.95. However, correlation-breaking anomalies were substantially more difficult because their individual feature values remained realistic.

Among the tested methods, LOF₂₀ provided the most consistent performance across both anomaly types. It achieved ROC AUC values of 0.9993 for global anomalies and 0.8886 for correlation-breaking

anomalies, while also obtaining the highest Precision@20 in both experiments. These results suggest that local density information is particularly useful when anomalous tracks are not globally extreme but instead contain unusual combinations of otherwise realistic features. Therefore, among the detectors evaluated in this assignment, LOF₂₀ is the most suitable choice for screening new tracks before they are added to playlists.